강화학습 Day1 HW1 레포트

로빛 예비단원 주호진

1) 과제

구분	과제 요건
구현	PyTorch MLP 구성, MNIST 학습·검증·테스트 분리 Forward, Loss, backward, step을 포함한 학습 루프 구현
결과	epoch별 학습·검증 Loss와 최종 테스트 Accuracy 제시 정답·예측값을 포함한 오분류 사례 및 원인 분석
제출물	GitHub 코드·실행 방법, 코드 설명 보고서 학습 로그·예측 결과와 실행 영상 MP4 제출
설명·준수	모델 구조·학습률 선택 이유 및 학습 루프 각 단계 설명 GPT 사용 시 사용 부분·사유 명시, 본인 코드 설명 필수

2) 코드

Ppt 에 있는 기본 코드를 토대로 구글링 + Gemini 사용으로 약간 수정했다.

```
batch_size = 256
learning_rate = 0.001
epochs = 20
```

학습 결과에 영향이 큰 파라미터 3 개값을 상단으로 올려 수정하기 편하게 설정했다.

```python
torch.manual_seed(0)
# 학습 데이터용: 데이터 증강
train_transform = transforms.Compose([
    transforms.RandomAffine(degrees=10, translate=(0.05, 0.05), scale=(0.95, 1.05)),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
full = datasets.MNIST(DATA_DIR, train=True, download=True, transform=train_transform)
train, val = random_split(full, [55000, 5000], generator=torch.Generator().manual_seed(0))
test = datasets.MNIST(DATA_DIR, train=False, download=True,
transform=test_transform)
train_loader = DataLoader(train, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val, batch_size=batch_size)
test_loader = DataLoader(test, batch_size=batch_size)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"device: {device}")
```

Accuracy 99% 이상이 나오게 하기 위해서 여러 방법을 찾아봤는데 98.8x%에서 증가하지 않아서 Gemini 를 사용하여 99%를 넘으려면 어떤 기법을 적용해야 하는지 물어본 뒤 데이터 증강을 사용하면 Accuracy 가 증가할 수 있다고 하여 데이터 증강을 적용했다.

RandomAffine 을 사용하여 이미지 각도, 이동, 확대 및 축소를 하여 픽셀 자체를 외우는 과적합을 방지해주도록 했다.

Testset 에서는 원본 그대로 판별해야하므로 데이터 증강이 적용되지 않도록 transform 을 분리해주었다.

추가적으로 MNIST 의 평균값과, 표준편차를 사용해 Z 스코어로 변환하는 정규화를 해주었다.

0~1 의 값을 가질 경우 가중치 기울기 방향이 하나로 통일되기 때문에, Z 점수로 변환하여 유연하게 움직일 수 있도록 해준다.

```
model = nn.Sequential(
nn.Flatten(),

nn.Linear(784, 1024),
nn.BatchNorm1d(1024),
nn.GELU(),
nn.Dropout(p=0.2),

nn.Linear(1024, 512),
nn.BatchNorm1d(512),
nn.GELU(),
nn.Dropout(p=0.10),

nn.Linear(512, 256),
nn.BatchNorm1d(256),
nn.GELU(),
nn.Dropout(p=0.1),

nn.Linear(256, 128),
nn.BatchNorm1d(128),
nn.GELU(),
nn.Dropout(p=0.05),

nn.Linear(128, 10)
).to(device)

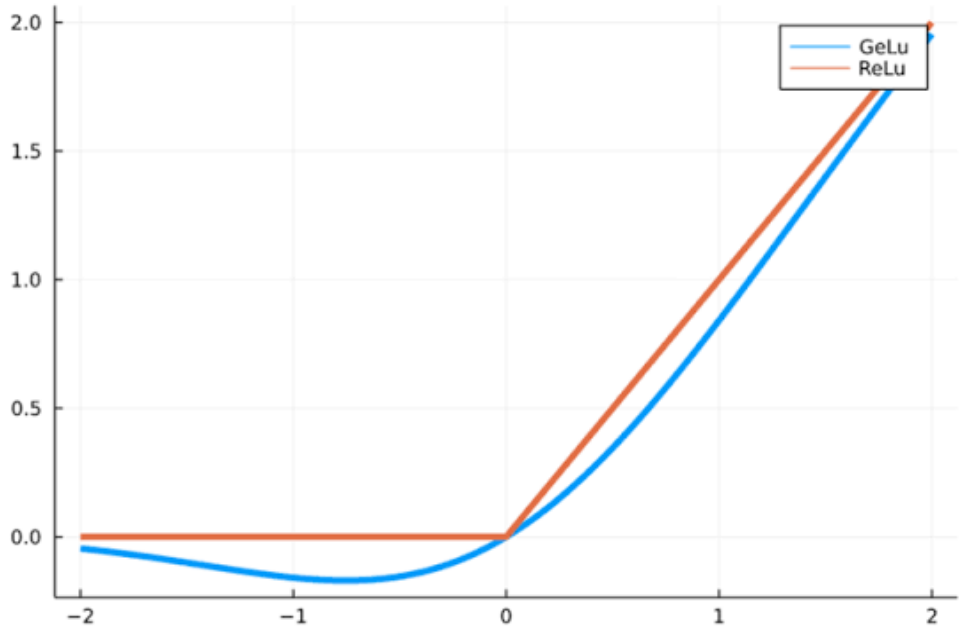
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity='relu')

model.apply(init_weights)
```

원래는 128 -> 128 -> 10, 256 -> 128 -> 128 -> 10 등 다양하게 바꿔보면서 과적합이 의심되는 경우 레이어나 노드를 줄이고, 일정 accuracy 에서 올라가지 않는 경우 레이어나 노드를 늘리는 등의 방식으로 바꾼 뒤에 학습을 계속 돌렸다. 여러 시도의 대부분에서 98.5% 부근 정체가 생겨 1024 까지 늘려서 적용하게 되었다.

BatchNorm1d 를 사용해 레이어마다 정규화를 다시 진행해주어 발산이나 소실을 방지한다.

처음에는 ppt 에 있는 ReLU 를 사용했는데, GELU 를 사용하면 0 부근의 애매한 값들을 좀 더 유연하게 처리해줄 수 있다고 하여 GELU 로 변경해서 학습해봤고 Accuracy 가 조금 더 높게 나와서 GELU 로 바꾸었다.



추가적으로 Dropout 을 사용하여 랜덤한 노드를 비활성화하여 과적합을 방지해주었고, 신경망의 가중치를 적절하게 초기화해주어 값이 소실되거나 발산하는 것을 방지해주는 Kaiming 초기화를 적용했다.

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=1e-4)
scheduler = lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=learning_rate,
    steps_per_epoch=len(train_loader),
    epochs=epochs
)
```

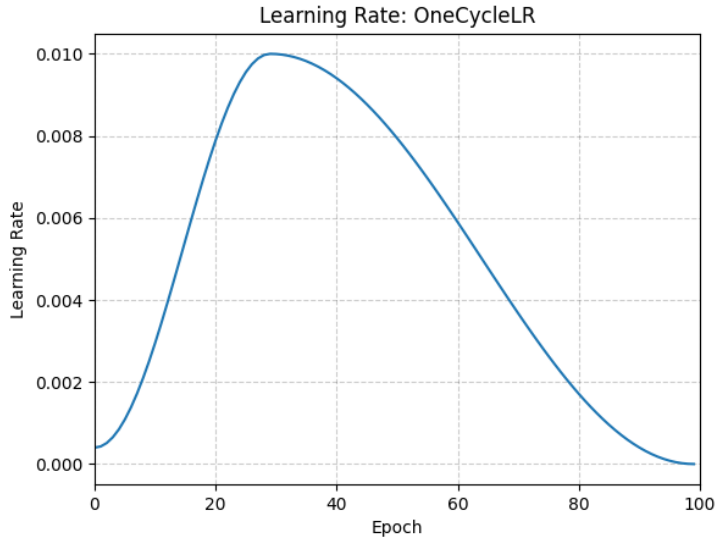
손실함수는 ppt 에 있는 cross entropy 를 사용했다.

cross entropy 란?

엔트로피를 서로 다른 확률 분포에서 계산한 값을 의미하며, 엔트로피는 불확실성을 수치로 나타낸 값이다. 값을 특정하기 어려울 수록(불확실성이 클 수록) 높은 값을 가진다.

옵티마이저는 ppt 에 있는 adam 을 쓰다가 가중치가 너무 커지는 것을 방지하는 weight_decay 를 고정값으로 설정했는데, weight_decay 을 설정하는 경우 AdamW 를 사용한다고 하여 AdamW 로 바꾸었다.

Adam 은 최근 변화가 컸던 가중치는 업데이트 폭을 줄이는데, 이때 weight_decay 도 같이 깎여나가 왜곡이 되는 부분을 분리하여 적용한 것이 AdamW 이다.



그리고 학습률을 고정하지 않고 가변적으로 바꿔주도록 했다. OneCycleLR 을 사용했으며 코사인함수 반주기와 비슷하게 학습률이 처음에 작은 값에서 시작하여 커졌다가 다시 작아지는 형태를 띤다. Max_lr, min_lr 등을 설정해줄 수 있다.

```python
def main():
    for epoch in range(epochs):
        model.train()
        total_loss = 0.0

        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            logits = model(images)
            loss = criterion(logits, labels)
            loss.backward()
            optimizer.step()
            scheduler.step()
            total_loss += loss.item() * images.size(0)

        train_loss = total_loss / len(train)
        val_loss, val_acc = evaluate(val_loader)
        print(f"\nepoch: {epoch + 1}")
        print(f"""train_loss: {train_loss}, LR: {scheduler.optimizer.state_dict()['param_groups'][0]['lr']}
              | Validation Loss: {val_loss}, Accuracy: {val_acc * 100:.2f}%""")

    test_loss, test_acc = evaluate(test_loader)
    print(f"\nTest Loss: {test_loss}, Accuracy: {test_acc * 100:.2f}%")
    visualize_wrong_sample(test_loader)
```

에포크마다 total_loss 를 0 으로 초기화하여 이전 에포크에 저장된 loss 는 지워주고, zero_grad 로 이전 배치에 사용된 기울기 정보를 0 으로 만든다.

그 다음 순전파 -> loss 함수로 오차를 구하고, 역전파를 통해 기울기 값을 수정해주고, 해당 기울기 값을 보고 옵티마이저에 따라 파라미터 값을 수정한다. 스케쥴러로 학습률을 조정하는

것을 반복한다.

한 에포크의 학습이 완료되면 loss 를 표시하고, 검증 set 의 loss 와 accuracy 를 확인해본다.

모든 학습이 끝나면 테스트셋으로 마지막 검증을 한다.

```python
def evaluate(loader):
    model.eval()
    loss_sum, correct, count = 0.0, 0, 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            logits = model(images)
            loss_sum += criterion(logits, labels).item() * len(labels)
            correct += (logits.argmax(1) == labels).sum().item()
            count += len(labels)

    return loss_sum / count, correct / count
```

검증 및 테스트 함수는 ppt 에 있는 함수를 그대로 사용했다.

model.eval()로 학습하지 않고 추론으로 전환하고, torch.no_grad()로 그래디언트 자동 미분을 비활성화한다.

```python
import math

import matplotlib.pyplot as plt


class WrongSampleViewer:···


def visualize_wrong_sample(test_loader):
    model.eval()
    wrong_samples = []

    # 전체 테스트 데이터에서 오답 추출
    with torch.no_grad():
        for images, labels in test_loader:
            images_dev = images.to(device)
            logits = model(images_dev)
            preds = torch.argmax(logits, dim=1).cpu()

            incorrect_mask = (preds != labels)
            for img, ans, pred in zip(images[incorrect_mask], labels[incorrect_mask], preds[incorrect_mask]):
                wrong_samples.append((img, ans.item(), pred.item()))

    print(f"\n총 틀린 샘플 개수: {len(wrong_samples)}개")

    # 키보드 조작 인터랙티브 뷰어 실행
    viewer = WrongSampleViewer(wrong_samples, page_size=25)
    viewer.show()
```

위 코드는 test 진행 시 틀린 숫자를 모아서 보여주는 코드이다. (visualization)

여기 코드를 레퍼런스로 넣어 Gemini 에게 입맛에 맞게 수정해달라고 한 뒤 추가했다.

3) 결과

영상

https://drive.google.com/file/d/1GRVHWeiAyeCOULPjgzsf54_175uOl3Gj/view?usp=sharing

epoch 20 일 때

```
epoch: 2
train_loss: 0.46235916400822724, LR: 0.0002803377920016139 | Validation Loss: 0.24691611881256104, Accuracy: 92.90%

epoch: 3
train_loss: 0.25136061451218344, LR: 0.0005205849356479811 | Validation Loss: 0.14761208546161653, Accuracy: 95.34%

epoch: 4
train_loss: 0.1755038484248248, LR: 0.000760675108572243 | Validation Loss: 0.11617995600700379, Accuracy: 96.36%

epoch: 5
train_loss: 0.1416664334600622, LR: 0.0009361787825957746 | Validation Loss: 0.11370549149513244, Accuracy: 96.28%

epoch: 6
train_loss: 0.1236247672774575, LR: 0.0009999997276640426 | Validation Loss: 0.09480613609552384, Accuracy: 96.94%

epoch: 7
train_loss: 0.11102152969837188, LR: 0.0009873476166062 | Validation Loss: 0.08531795260906219, Accuracy: 97.18%

epoch: 8
train_loss: 0.09840215588699687, LR: 0.0009502579613536496 | Validation Loss: 0.08367139456272125, Accuracy: 97.54%

epoch: 9
train_loss: 0.08696272493600846, LR: 0.0008905905920936668 | Validation Loss: 0.08126781637668609, Accuracy: 97.64%

epoch: 10
train_loss: 0.080358558604934, LR: 0.0008113374798701835 | Validation Loss: 0.06978768711090087, Accuracy: 97.78%

epoch: 11
train_loss: 0.07273765675046227, LR: 0.0007164727066622615 | Validation Loss: 0.061936562383174895, Accuracy: 98.00%

epoch: 12
train_loss: 0.0666565396417271, LR: 0.0006107531883193416 | Validation Loss: 0.060639054965972904, Accuracy: 98.00%

epoch: 13
train_loss: 0.06440389071486213, LR: 0.000499480142937402 | Validation Loss: 0.055413458251953124, Accuracy: 98.34%

epoch: 14
train_loss: 0.0565641085868532, LR: 0.0003883823326564766633 | Validation Loss: 0.05078762222528457, Accuracy: 98.42%

epoch: 15
train_loss: 0.05114894925355911, LR: 0.0002825903939403946 77 | Validation Loss: 0.044364509230852125, Accuracy: 98.56%

epoch: 16
train_loss: 0.04611811967329545, LR: 0.0001878505115680416 | Validation Loss: 0.04638017443418503, Accuracy: 98.54%

epoch: 17
train_loss: 0.043409487797997214, LR: 0.00010876266279321758 | Validation Loss: 0.04479648705720902, Accuracy: 98.46%

epoch: 18
train_loss: 0.04192365287596529, LR: 4.9293188059140435e-05 | Validation Loss: 0.03860680941939354, Accuracy: 98.78%

epoch: 19
train_loss: 0.04118268641992049, LR: 1.2424135151877813e-05 | Validation Loss: 0.04096006486415863, Accuracy: 98.64%

epoch: 20
train_loss: 0.039080977639014074, LR: 4.27233595728802e-09 | Validation Loss: 0.042309683257341385, Accuracy: 98.90%

Test Loss: 0.023494386935979127, Accuracy: 99.28%

총 틀린 샘플 개수: 72개
```

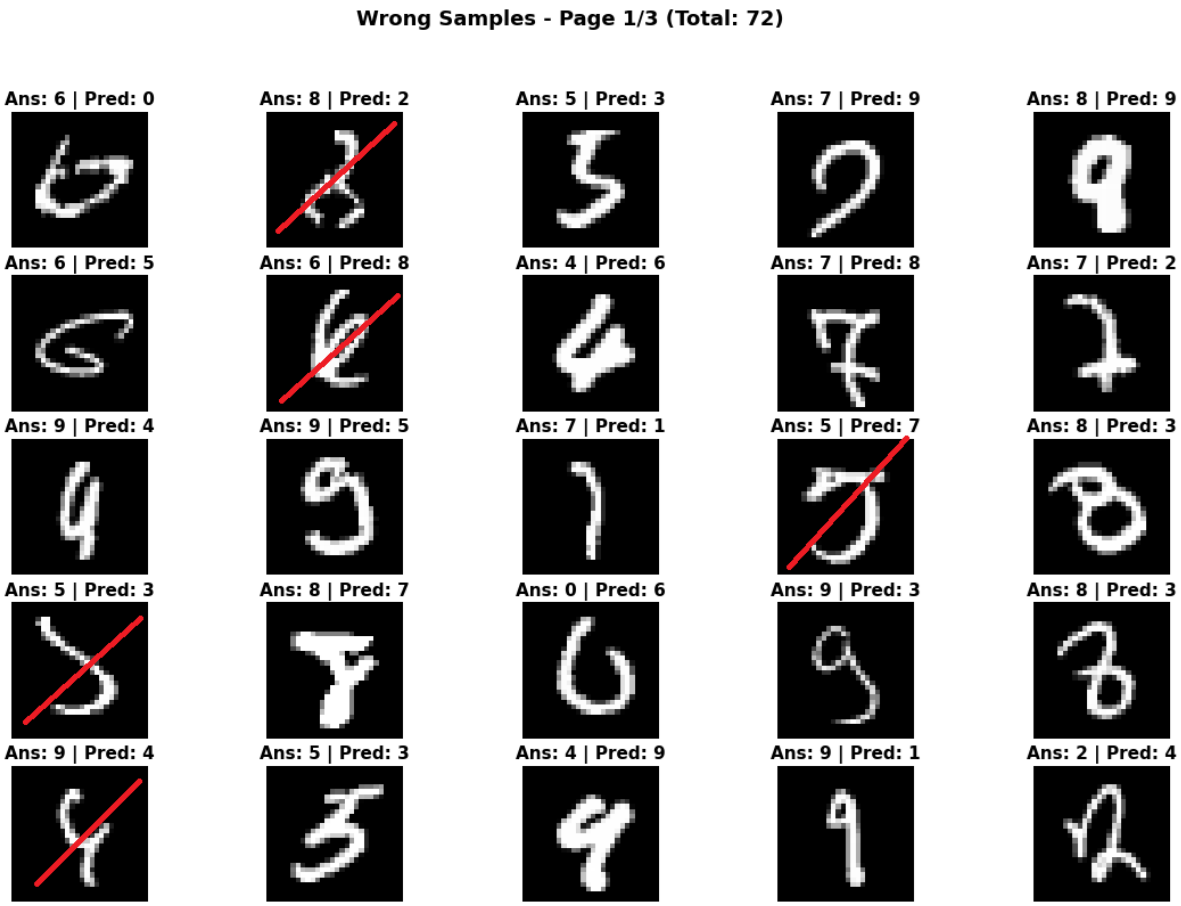
Learning late

0.01, 0.001, 0.0001 중 0.01 과 0.001 은 결과가 비슷하여 0.001 사용, 0.0001 은 정확도가 하락함

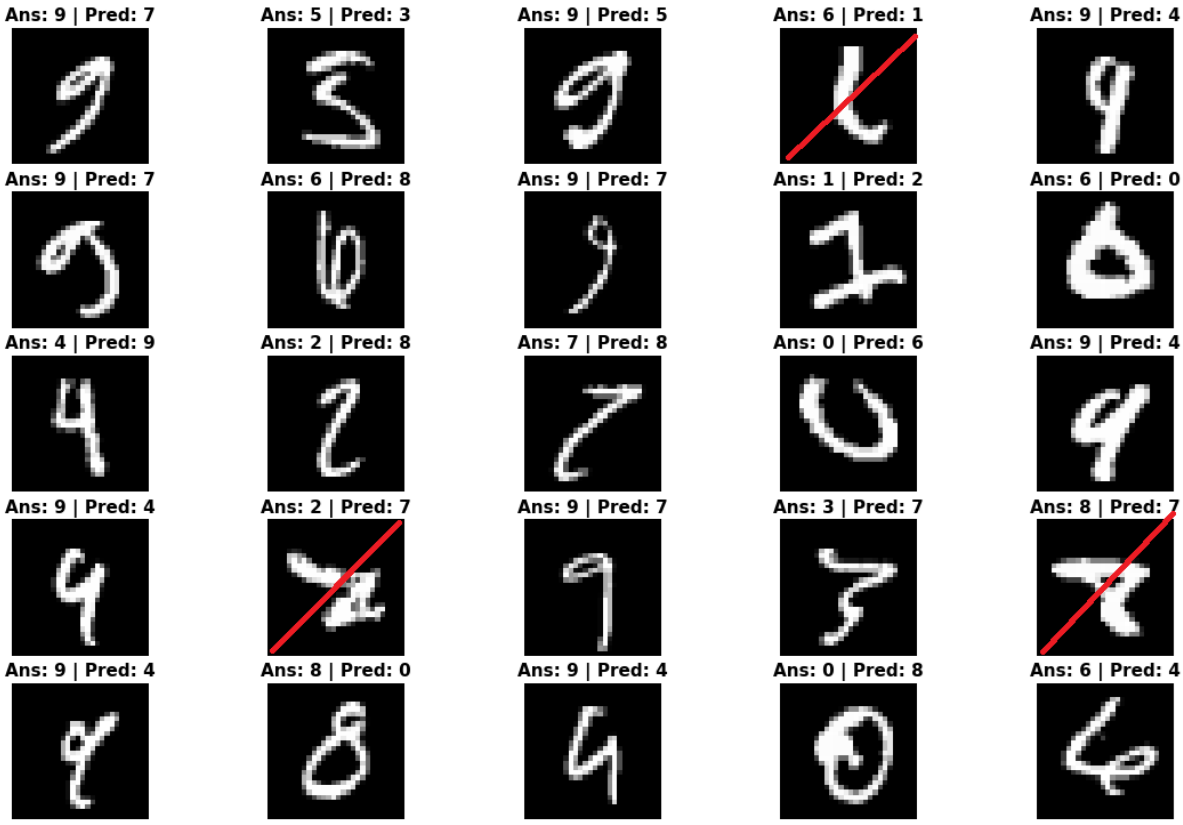
Batch size

Cuda 를 사용하여 batch size 를 키웠다. 128 미만은 256 과 차이가 유의미하게 나지 않아서 학습 속도를 위해 256 을 선택했다, 512 부터는 정확도가 하락하는 모습을 보였다.

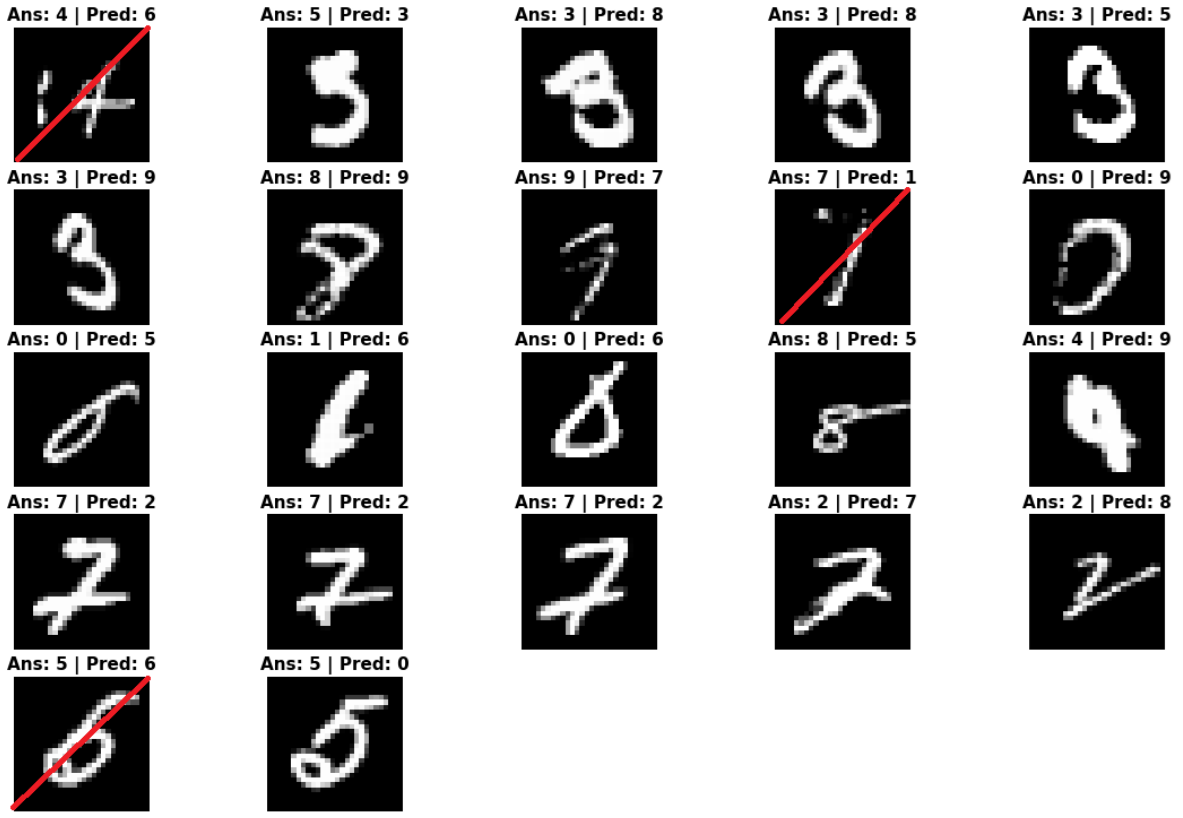
오답 케이스 분석



Wrong Samples - Page 2/3 (Total: 72)



Wrong Samples - Page 3/3 (Total: 72)



사람 눈으로 봤을 때도 헷갈리는 몇 데이터를 제외하고 봤을 때 공통적으로 보이는 케이스는 다음과 같다.

숫자의 동그라미 부분이 완전히 닫히지 않은 경우(6, 8, 9, 0 의 숫자에서 원형이 열려있는 경우)

7 을 쓸 때 가운데 가로 획이 있는 경우

3 과 5 를 구분하기 애매한 경우

꼬리가 왼쪽 또는 오른쪽으로 길게 빠지는 경우

---

epoch 줄였을 때(15)

```
epoch: 1
train_loss: 1.2289714732083408, LR: 0.0001525216294230912 | Validation Loss: 0.49873064699172975, Accuracy: 86.06%

epoch: 2
train_loss: 0.39531940228288825, LR: 0.0004373318631589169 6 | Validation Loss: 0.1999248749256134, Accuracy: 93.90%

epoch: 3
train_loss: 0.21508984165408396, LR: 0.0007609002363061166 | Validation Loss: 0.12952595300674438, Accuracy: 95.96%

epoch: 4
train_loss: 0.15957356954271143, LR: 0.0009715249131300999 | Validation Loss: 0.11032259771823882, Accuracy: 96.46%

epoch: 5
train_loss: 0.13034390299536966, LR: 0.0009943112517080577 | Validation Loss: 0.1040352609038353, Accuracy: 96.34%

epoch: 6
train_loss: 0.11207603552991693, LR: 0.0009501822954580514 | Validation Loss: 0.08729576363563538, Accuracy: 97.18%

epoch: 7
train_loss: 0.10113029724684629, LR: 0.0008660528443729102 | Validation Loss: 0.07897600793838501, Accuracy: 97.40%

epoch: 8
train_loss: 0.08901502594405955, LR: 0.0007493981693775642 | Validation Loss: 0.07845824891328812, Accuracy: 97.78%

epoch: 9
train_loss: 0.07783937612988732, LR: 0.0006105835502759591 | Validation Loss: 0.0733446922302246, Accuracy: 97.66%

epoch: 10
train_loss: 0.07091361931562423, LR: 0.0004619432751531945 3 | Validation Loss: 0.062175652813911436, Accuracy: 97.98%

epoch: 11
train_loss: 0.06303172418800267, LR: 0.0003166846847510317 | Validation Loss: 0.05780104378461838, Accuracy: 98.32%

epoch: 12
train_loss: 0.05640430068536238, LR: 0.0001877146422835272 | Validation Loss: 0.05510139414072037, Accuracy: 98.24%

epoch: 13
train_loss: 0.05584775235652924, LR: 8.649270199962813e-05 | Validation Loss: 0.050931843703985215, Accuracy: 98.46%

epoch: 14
train_loss: 0.049939446182142605, LR: 2.2012877498727224e-05 | Validation Loss: 0.046434862172603605, Accuracy: 98.58%

epoch: 15
train_loss: 0.04818113742741671, LR: 4.48415277875558e-09 | Validation Loss: 0.04324843672215939, Accuracy: 98.48%

Test Loss: 0.026108733589353506, Accuracy: 99.13%

총 틀린 샘플 개수: 87개
```

epoch 늘렸을 때(25)

```
epoch: 8
train_loss: 0.10632170848196203, LR: 0.0009979495583280775 | Validation Loss: 0.08857569727897643, Accuracy: 97.40%

epoch: 9
train_loss: 0.09398154757781463, LR: 0.0009818702678356052 | Validation Loss: 0.08585704579353333, Accuracy: 97.46%

epoch: 10
train_loss: 0.08657593241930008, LR: 0.0009503033347323649 | Validation Loss: 0.07325714243650436, Accuracy: 97.64%

epoch: 11
train_loss: 0.07914915471185337, LR: 0.0009042633462216541 | Validation Loss: 0.06555999151468277, Accuracy: 98.06%

epoch: 12
train_loss: 0.0736562721750953, LR: 0.0008452300654160359 | Validation Loss: 0.06511331126689911, Accuracy: 97.80%

epoch: 13
train_loss: 0.07094763291423971, LR: 0.0007751008705333478 | Validation Loss: 0.06015679793357849, Accuracy: 98.14%

epoch: 14
train_loss: 0.06456092887629163, LR: 0.0006961297715996132 | Validation Loss: 0.058817809575796125, Accuracy: 98.22%

epoch: 15
train_loss: 0.059120704742995175, LR: 0.0006108549647121991 | Validation Loss: 0.051327312183380124, Accuracy: 98.44%

epoch: 16
train_loss: 0.053161653410846534, LR: 0.0005220172523288117 | Validation Loss: 0.051389696979522705, Accuracy: 98.40%

epoch: 17
train_loss: 0.0500036212888631, LR: 0.00043247195162136653 | Validation Loss: 0.05350424919724464, Accuracy: 98.30%

epoch: 18
train_loss: 0.04813195123997602, LR: 0.0003450971222327194 | Validation Loss: 0.042872847253084184, Accuracy: 98.60%

epoch: 19
train_loss: 0.04454325123429299, LR: 0.0002627010630716705 6 | Validation Loss: 0.044031607279181484, Accuracy: 98.40%

epoch: 20
train_loss: 0.04022774729891257, LR: 0.0001879320512753595 6 | Validation Loss: 0.043413315677642825, Accuracy: 98.76%

epoch: 21
train_loss: 0.03862218868895011, LR: 0.0001231932244030674 | Validation Loss: 0.05017333411574364, Accuracy: 98.48%

epoch: 22
train_loss: 0.035462670960751445, LR: 7.056534161775417e-05 | Validation Loss: 0.04253710836172104, Accuracy: 98.72%

epoch: 23
train_loss: 0.03302857582433657, LR: 3.173990637450584e-05 | Validation Loss: 0.04159080362319946, Accuracy: 98.56%

epoch: 24
train_loss: 0.03213036476211115, LR: 7.964800107714775e-06 | Validation Loss: 0.040166414940357206, Accuracy: 98.78%

epoch: 25
train_loss: 0.031710130329565565, LR: 4.174295018344211e-09 | Validation Loss: 0.04531053614020348, Accuracy: 98.68%

Test Loss: 0.02312915639344137, Accuracy: 99.33%

총 틀린 샘플 개수: 67개
```

Epoch 15 -> 20 에서는 0.15%p 가 올라간 반면 20 -> 25 에서 0.05%p 만 향상되었기도 하고, 검증 accuracy 가 확실히 개선된다고 보기 어려워지는 지점이라 생각해 20 이 최적이라고 판단하고 20 을 사용했다.

+ learning late=0.01, epoch=25 일 때 99.38% (seed=1 -> 99.29%, seed=777 -> 99.19%)

https://drive.google.com/file/d/12JIqC_I4dlwWibqGAscCY9XPo5A2HrwD/view?usp=sharing

```
epoch: 9
train_loss: 0.08407609974904494, LR: 0.009818702678356052 | Validation Loss: 0.0902063783288002, Accuracy: 97.16%

epoch: 10
train_loss: 0.07920633987188339, LR: 0.00950303334732365 | Validation Loss: 0.06762897743582726, Accuracy: 97.72%

epoch: 11
train_loss: 0.0715820064718073, LR: 0.009042633462216542 | Validation Loss: 0.07043990635871887, Accuracy: 97.76%

epoch: 12
train_loss: 0.06553249951817773, LR: 0.008452300654160359 | Validation Loss: 0.06792973853349686, Accuracy: 97.86%

epoch: 13
train_loss: 0.06399152587327091, LR: 0.0077510087053334785 | Validation Loss: 0.06186914377808571, Accuracy: 98.12%

epoch: 14
train_loss: 0.05782358973730694, LR: 0.006961297715996133 | Validation Loss: 0.05321186112165451, Accuracy: 98.30%

epoch: 15
train_loss: 0.051609578704833986, LR: 0.006108549647121992 | Validation Loss: 0.05268140704631805, Accuracy: 98.42%

epoch: 16
train_loss: 0.046603160521117126, LR: 0.005220172523288117 | Validation Loss: 0.04917163009643555, Accuracy: 98.66%

epoch: 17
train_loss: 0.04148416228511117, LR: 0.004324719516213665 | Validation Loss: 0.053476885980367664, Accuracy: 98.48%

epoch: 18
train_loss: 0.03719745403595946, LR: 0.003450971222327194 | Validation Loss: 0.04648126268982887, Accuracy: 98.56%

epoch: 19
train_loss: 0.03542213937260888, LR: 0.002627010630716706 | Validation Loss: 0.04248274813294411, Accuracy: 98.62%

epoch: 20
train_loss: 0.02932308730591427, LR: 0.0018793205127535957 | Validation Loss: 0.04264295290112496, Accuracy: 98.72%

epoch: 21
train_loss: 0.027868025036562574, LR: 0.0012319322440306742 | Validation Loss: 0.043953576403856276, Accuracy: 98.70%

epoch: 22
train_loss: 0.024174635050242598, LR: 0.0007056534161775417 | Validation Loss: 0.04396444925665855, Accuracy: 98.86%

epoch: 23
train_loss: 0.0207056717552922, LR: 0.0003173990637450584 | Validation Loss: 0.03511402575969696, Accuracy: 98.94%

epoch: 24
train_loss: 0.019538904345306484, LR: 7.964800107714776e-05 | Validation Loss: 0.03594446358084678, Accuracy: 98.88%

epoch: 25
train_loss: 0.01975134329944849, LR: 4.1742950183442107e-08 | Validation Loss: 0.042874482840299606, Accuracy: 98.64%

Test Loss: 0.020976797322199855, Accuracy: 99.38%

총 틀린 샘플 개수: 62개
```

```
epoch: 25
train_loss: 0.020959725038029932, LR: 4.174295018344

Test Loss: 0.021366317817618257, Accuracy: 99.29%
```

```
epoch: 25
train_loss: 0.020388202020864596, LR: 4.174295018344

Test Loss: 0.022761229665498833, Accuracy: 99.19%
```